Core Philosophy

> **“அவையறிந்து ஆராய்ந்து சொல்லுக சொல்லின்**
> **தொகையறிந்த தூய்மை யவர்″**
>
> — திருக்குறள்

Systems Report

LGC Articulate DevLang

Intelligent Communication Training
and Evaluation System

AI-Assisted Articulation • Structured Thinking • Contextual Communication

Ramalingam Jayavelu

Founder & Builder — LGC Systems

CHAPTER I

INTRODUCTION

1.1. Introduction

Communication plays an important role in professional environments, academic discussions, technical collaborations, and day-to-day interactions. The ability to express ideas clearly, organize thoughts effectively, and communicate with confidence has become an essential skill in modern digital and professional ecosystems. However, many individuals face difficulties in structuring responses, maintaining clarity, and articulating their thoughts properly during communication.

With the advancement of Artificial Intelligence and intelligent software systems, modern applications are increasingly capable of analyzing user input, generating guided feedback, and assisting users in improving communication quality. AI-assisted systems can help users understand how to structure responses, improve articulation, and develop better communication habits through automated evaluation and guided refinement mechanisms.

LGC Articulate: Intelligent Communication Training and Evaluation System is developed as a full-stack AI-assisted platform focused on improving articulation, structured thinking, and communication skills. The system provides users with an interactive environment where they can learn, evaluate, and refine their responses using intelligent processing techniques and guided feedback workflows.

The platform consists of three major modules namely Learn Mode, Evaluate Mode, and Doubt Mode. Learn Mode provides guided scenario-based

articulation training to help users understand how to approach communication systematically. Evaluate Mode performs AI-assisted response evaluation and generates structured feedback including scores, strengths, gaps, and improvement suggestions. Doubt Mode helps users refine uncertain responses through coaching-oriented AI guidance.

The frontend of the system is developed using React and Vite to provide a modular and responsive user interface, while the backend is implemented using Node.js, Express.js, and MongoDB for secure data handling and scalable request processing. JWT authentication is integrated for secure access and user-specific evaluation history management. The system also incorporates AI orchestration and validation layers to ensure structured and reliable response generation.

This project aims to create an intelligent communication training environment that combines modern full-stack technologies with AI-assisted evaluation techniques to help users improve articulation quality, structured thinking, and communication effectiveness through guided and automated workflows.

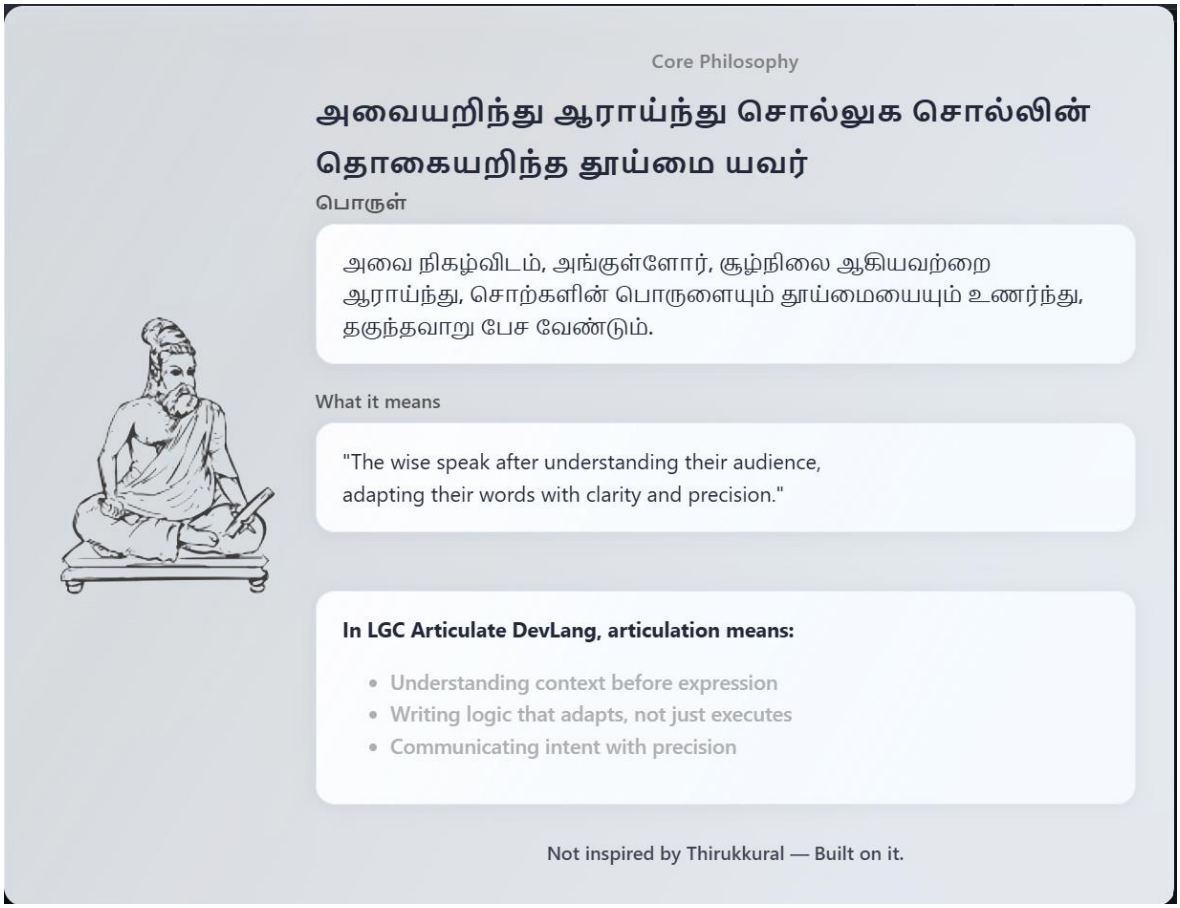

Fig 1.1 Core Philosophy of Articulation

The above figure illustrates the foundational communication philosophy behind the proposed system, which is not merely inspired by, but built on

Thirukkural – Porutpaal / Amaichiyal / Avaiyarithal – Kural 711

The kural emphasizes understanding the audience, context, and clarity before speaking, which directly aligns with the core objective of the project. Based on this principle, the system was designed to help users improve articulation quality, structured thinking, contextual communication, and professional response generation through AI-assisted evaluation and guided communication workflows.

1.2. Problem Statement

The development of an intelligent communication training and evaluation system involves several challenges in creating a reliable and effective platform for users. Many existing communication systems focus mainly on conversation and response generation rather than helping users improve articulation, structured thinking, and communication clarity. Users often face difficulty in organizing thoughts, structuring responses, and communicating effectively in professional and academic environments.

The integration of Artificial Intelligence for response evaluation introduces additional complexity in generating structured feedback, maintaining response consistency, and providing meaningful communication guidance. The system must also ensure secure user management, efficient evaluation workflows, and an interactive user experience.

1.3. Objective

The main goal of this project is to develop an intelligent communication training and evaluation system that helps users improve articulation, structured thinking, and communication quality. The system aims to provide guided learning, AI-assisted response evaluation, and communication refinement through an interactive full-stack platform.

The project also focuses on integrating modern frontend and backend technologies with AI-based evaluation techniques to create a scalable, secure, and user-friendly communication training environment.

1.4. Scope of the System

The scope of this project is to develop an intelligent communication training and evaluation platform that assists users in improving articulation, structured thinking, and communication quality. The system provides guided communication learning, AI-assisted response evaluation, and coaching-oriented refinement through an interactive full-stack application.

The platform is designed to support scenario-based communication practice, structured feedback generation, and user-specific evaluation history management. The system focuses on improving communication effectiveness in professional, academic, and technical environments through automated evaluation workflows and guided articulation training.

The project also includes secure user authentication, frontend-backend integration, database management, and AI orchestration mechanisms to ensure reliable system performance and user interaction. The platform can be further extended with advanced analytics, mobile support, improved AI evaluation models, and enhanced user experience features in future developments.

CHAPTER II

LITERATURE REVIEW

The design and implementation of automated systems for enhancing technical communication are explored across several research domains. In the article [1], “Use of Natural Language Processing (NLP) Tools to Assess Digital Literacy Skills,” the authors develop a framework using text mining and semantic analysis to evaluate communication competencies. This research proves that NLP can reduce human bias in assessment and provide automated feedback based on consistent semantic patterns.

In the article [2], “Automated Soft Skills Assessment Using Natural Language Processing and Machine Learning Techniques,” the researchers propose a machine learning model to assess soft skills such as confidence and clarity. By integrating NLP for content analysis with linguistic patterns, the system evaluates communication effectiveness and provides structured feedback to users.

In the article [3], “The Effect of Using a Conversational Agent for Practicing Job Interviews: A Randomized Controlled Trial,” a chatbot-based interview simulator was developed to help users practice professional communication scenarios. The study highlights that AI-assisted communication training significantly improves response quality and reduces communication anxiety among learners.

In the article [4], “An Intelligent Feedback System for Enhancing Communication Skills through NLP and Machine Learning,” an intelligent feedback engine was designed to evaluate linguistic patterns and generate structured communication guidance. The authors observed that AI-assisted

feedback systems help users improve articulation quality and structured response generation.

In the article [5], “Context-Aware Response Evaluation for Intelligent Tutoring Systems using Transformer Models,” the research introduces context-aware AI models capable of evaluating user responses based on scenario understanding and semantic relevance. This approach improves response evaluation accuracy by considering communication context rather than relying only on grammar or keyword matching.

These research studies demonstrate that Artificial Intelligence, Natural Language Processing, and intelligent evaluation systems can significantly improve communication training and automated response assessment. The proposed system, LGC Articulate: Intelligent Communication Training and Evaluation System, builds upon these concepts by integrating AI-assisted evaluation, guided articulation training, and structured communication refinement within a unified full-stack platform.

CHAPTER III

SYSTEM DESIGN AND DEVELOPMENT

3.1. System Overview

LGC Articulate: Intelligent Communication Training and Evaluation System is a full-stack AI-assisted platform developed to improve articulation, structured thinking, and communication skills. The system follows a workflow consisting of user input, AI evaluation, guided feedback generation, and response refinement.

The platform includes Learn Mode, Evaluate Mode, and Doubt Mode for communication training, response evaluation, and refinement workflows. The frontend is developed using React and Vite, while the backend uses Node.js, Express.js, and MongoDB. JWT authentication and AI orchestration mechanisms are integrated for secure and structured system operation.

TABLE 3.1

TECHNOLOGY STACK

S.NO	TECHNOLOGY/ TOOL	PURPOSE
1	React	Frontend user interface development
2	Vite	Frontend build tool and development server
3	React Router	Client-side routing and navigation
4	Node.js	Backend runtime environment
5	Express.js	Backend API and server framework

6	MongoDB	Database management system
7	Mongoose	MongoDB object modeling and schema management
8	JSON Web Token (JWT)	User authentication and protected route access
9	Helmet	Backend security middleware
10	CORS	Cross-origin request handling
11	Express Rate Limit	API request rate limiting
12	Morgan	HTTP request logging
13	dotenv	Environment variable management
14	OpenRouter / Gemini	AI-assisted communication evaluation and doubt clarification
15	Visual Studio Code	Source code development environment

3.2. Frontend Architecture

The frontend is developed using React and Vite with a feature-based architecture for modular development. Core application files include src/main.jsx, src/app/App.jsx, and src/app/Routes.jsx for application bootstrap and routing.

Modules such as Learn, Evaluate, Doubt, and Authentication are organized inside separate feature directories. The frontend communicates with the backend using the services/api.js abstraction layer and uses React Router for navigation. LocalStorage is used for token management and usage tracking.

3.3. Backend Architecture

The backend is developed using Node.js and Express.js and follows a layered architecture consisting of routes, middleware, controllers, services, and models. The main backend initialization is handled through server.js, which loads environment variables, applies middleware, connects to MongoDB, and mounts API routes.

```
const app = express();

// ==================== MIDDLEWARE ====================

app.use(helmet());

app.use(cors({
  origin: config.corsOrigin,
  credentials: true
}));

app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

if (config.nodeEnv === 'development') {
  app.use(morgan('dev'));
} else {
  app.use(morgan('combined'));
}

const limiter = rateLimit({
  windowMs: config.rateLimitWindowMs,
  max: config.rateLimitMaxRequests,
  message: {
    success: false,
    error: {
      message: 'Too many requests. Please try again later.'
    }
  },
  standardHeaders: true,
  legacyHeaders: false
});

app.use('/api', limiter);
```

Fig 3.1 Backend Middleware Configuration

The above code snippet illustrates the backend middleware configuration implemented using Express.js. Security middleware such as Helmet, CORS handling, request parsing, logging, and API rate limiting are integrated to improve backend security, request management, and scalable API processing.

Route files such as authRoutes.js, evaluationRoutes.js, and doubtRoutes.js handle endpoint mapping, while middleware files manage authentication, validation, and error handling. Service files including evaluationService.js and aiRefinementService.js handle AI orchestration and evaluation workflows.

MongoDB persistence is managed through Mongoose models such as User.js and attemptModel.js.

3.4. Database Design

MongoDB is used as the primary database for storing user information, authentication details, evaluation history, and AI-generated feedback records. Mongoose ODM is used for schema management and database interaction within the backend. Core database models include User.js and attemptModel.js for user management and evaluation persistence.

3.5. Authentication System

The system uses JWT-based authentication for secure access to protected modules and API routes. User registration, login, verification, and password reset functionalities are handled through authRoutes.js and authController.js. Password hashing and token verification mechanisms are integrated to improve security and protected route access.

3.6. AI Evaluation Pipeline

The AI evaluation pipeline processes user responses and generates structured communication feedback. User input is forwarded through controllers and services to aiRefinementService.js, where AI orchestration and evaluation workflows are handled. Validation utilities such as aiGuard.js and defensiveValidator.js are used to maintain structured and reliable output generation.

```
export async function evaluateAPI(input) {
  try {
    const res = await apiFetch("/api/evaluate", {
      method: "POST",
      body: JSON.stringify(input)
    });

    const data = await safeParse(res);

    console.log("RAW API RESPONSE:", data);

    if (!res.ok) {
      throw new Error(data?.error || "API error");
    }
```

Fig 3.2 AI Evaluation Request Handling

The above code snippet illustrates the AI evaluation request workflow implemented in the frontend application. The evaluateAPI() function sends user responses to the backend evaluation endpoint and processes the structured feedback returned from the AI evaluation pipeline. The implementation also includes response validation and error handling mechanisms to improve reliability during communication evaluation workflows.

3.7. Learn Mode Module

Learn Mode provides guided communication training through scenario-based articulation workflows. The module helps users understand structured response building and communication clarity using interactive learning activities. Frontend learning workflows are managed through the features/learn module within the React application architecture.

3.8. Evaluate Mode Module

Evaluate Mode is responsible for AI-assisted response evaluation and structured feedback generation. Users submit responses based on communication scenarios, and the system evaluates articulation quality, clarity, and response structure. Evaluation workflows are managed through features/evaluate, evaluationRoutes.js, and evaluationService.js.

3.9. Doubt Mode Module

Doubt Mode helps users refine uncertain or incomplete responses through AI-assisted coaching workflows. Users provide communication context and responses, which are processed through doubtRoutes.js and AI refinement services to generate guided communication suggestions. Frontend interaction handling is managed through the features/doubt module.

3.10. API Integration

The frontend and backend communicate through REST API integration using JSON-based request and response workflows. API communication is managed through services/api.js, which handles token injection, request configuration, and response processing. Protected routes use JWT authentication for secure access to evaluation and history-related functionalities.

TABLE 3.2

API ENDPOINTS

S.NO	METHOD	ENDPOINT	PURPOSE
1	POST	/api/auth/signup	User registration
2	POST	/api/auth/login	User authentication and JWT generation
3	POST	/api/auth/forgot-password	Password reset request
4	POST	/api/auth/reset-password	Password reset functionality
5	POST	/api/evaluate	AI-assisted response evaluation
6	POST	/api/evaluate/test	Public evaluation testing
7	GET	/api/evaluate/user/history	Fetch user evaluation history
8	GET	/api/evaluate/:attemptId	Fetch specific evaluation attempt
9	POST	/api/doubt	AI-assisted doubt refinement
10	GET	/health	Backend health status check

```
// client/src/services/api.js

const BASE_URL =
  import.meta.env.VITE_SERVER_URL || "http://localhost:3000";

function getToken() {
  return localStorage.getItem("token");
}

async function apiFetch(url, options = {}) {
  const token = getToken();

  try {
    return await fetch(`${BASE_URL}${url}`, {
      ...options,
      headers: {
        "Content-Type": "application/json",
        ...(token && {
          Authorization: `Bearer ${token}`
        }),
        ...options.headers
      }
    });
```

Fig 3.3 Frontend API Abstraction using services/api.js

The above code snippet demonstrates the centralized API abstraction layer implemented in the frontend using services/api.js. The module manages backend communication by configuring the base server URL, handling request headers, and automatically attaching JWT authentication tokens to protected API requests. This approach improves maintainability and ensures consistent communication between the frontend and backend systems.

3.11. Environment Configuration

The system uses environment-based configuration for managing database connections, JWT secrets, API keys, and frontend-backend integration settings. Environment variables are loaded using dotenv and validated through configuration utilities inside config/index.js. This approach improves security, maintainability, and deployment flexibility across development and production environments.

3.12. System Workflow

The system workflow begins with user authentication through registration or login processes. After successful authentication, users can access Learn Mode, Evaluate Mode, and Doubt Mode functionalities. User responses are processed through backend APIs and forwarded to AI orchestration services for evaluation and feedback generation. The generated results are returned to the frontend and stored in MongoDB for future reference and history tracking.

3.13. Overall Architecture Diagram

The overall architecture of the system consists of frontend, backend, database, and AI orchestration layers. The React frontend communicates with the Node.js and Express.js backend through REST APIs. MongoDB is used for storing user data and evaluation history, while AI orchestration services process user responses and generate structured communication feedback. The architecture is designed to support modular development, secure communication, and scalable workflow management.

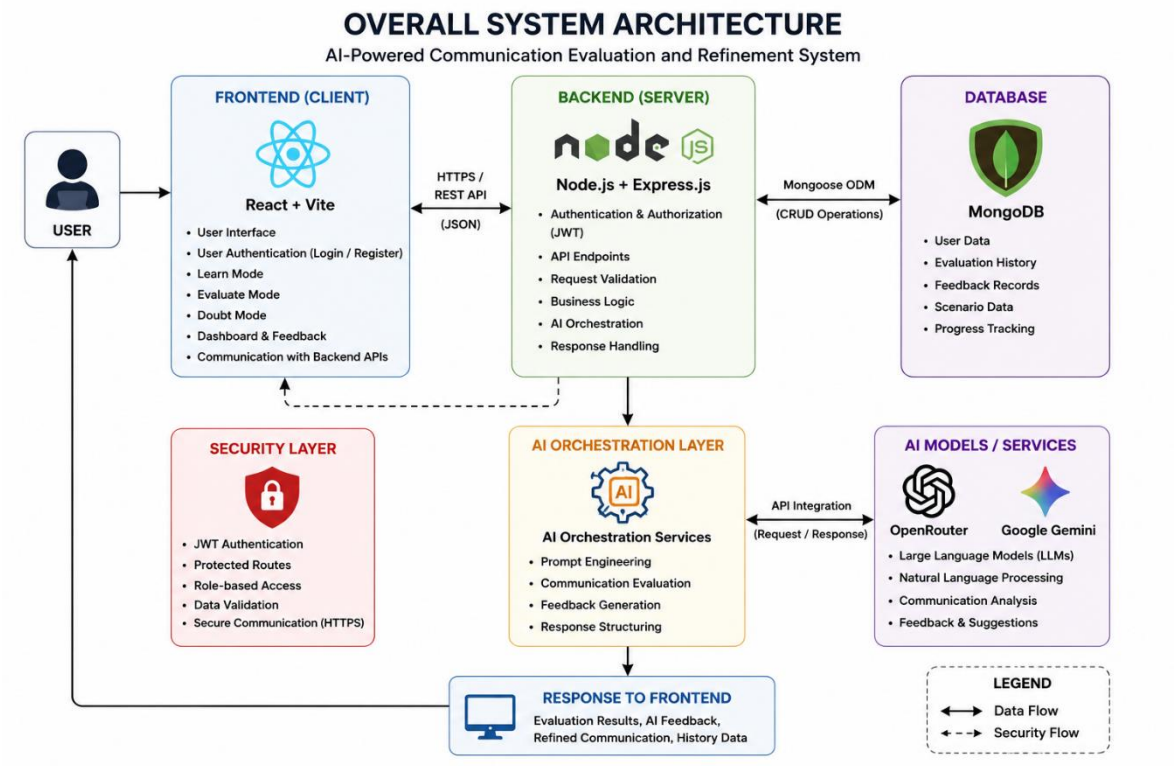

Fig 3.4 Overall System Architecture

The above figure illustrates the overall architecture of the proposed system. The React frontend communicates with the Node.js and Express.js backend through REST APIs, while MongoDB handles data persistence and AI orchestration services process communication evaluation and refinement workflows.

CHAPTER IV
IMPLEMENTATION AND WORKING PRINCIPLE

4.1. User Authentication Workflow

The system implements JWT-based authentication to provide secure access to protected functionalities. Users can register and log in using email and password credentials through the authentication module. After successful login, the backend generates a JWT token which is stored in LocalStorage and attached to protected API requests for authorization.

Authentication-related workflows are handled through authRoutes.js, authController.js, and authentication middleware integrated within the backend architecture. The system also supports password reset and account verification functionalities for secure user management.

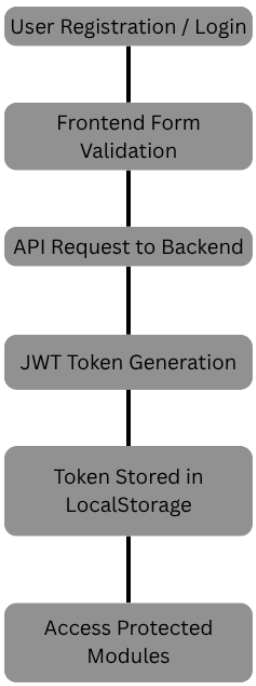

Fig 4.1 User Authentication Workflow

The above flowchart illustrates the authentication workflow of the system. Users submit registration or login credentials through the frontend interface, after which the backend validates the request and generates a JWT token for secure access to protected application modules.

4.2. Communication Evaluation Workflow

The communication evaluation workflow begins when a user submits a response through the Evaluate Mode interface. The frontend sends the response data along with communication context to the backend evaluation endpoint using REST API requests.

The backend processes the request through middleware, controllers, and AI orchestration services. The AI evaluation pipeline analyzes articulation quality, response structure, and communication clarity before generating structured feedback including scores, strengths, gaps, and improvement suggestions. The generated evaluation data is then returned to the frontend and stored in MongoDB for future reference.

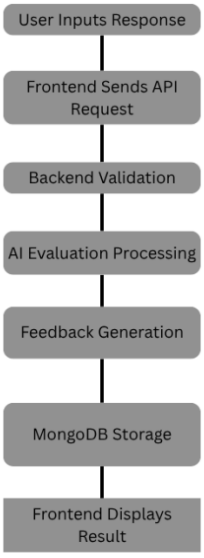

Fig 4.2 Communication Evaluation Workflow

The above flowchart illustrates the communication evaluation workflow implemented in the system. User responses are processed through frontend-backend communication, backend validation, AI evaluation services, and database storage before structured feedback is displayed to the user interface.

4.3. AI Response Processing

The system uses AI orchestration services to process user responses and generate structured communication feedback. User input is forwarded to AI refinement services where the communication context and response data are analyzed using predefined prompt workflows and validation mechanisms.

The generated AI output is processed using validation utilities such as aiGuard.js and defensive validation logic to maintain output consistency and reliability. The processed response is then formatted and returned to the frontend for structured result rendering and user interaction.

4.4. Database Operations

The system uses MongoDB for storing user information, authentication details, evaluation history, and AI-generated feedback records. Database interactions are managed using Mongoose models and service-layer workflows within the backend architecture.

When users submit communication responses, the evaluation results and related metadata are processed and stored in the database for future access. The system also supports retrieval of evaluation history and individual attempt details through protected API endpoints.

4.5. Frontend-Backend Communication

The frontend and backend communicate using REST API-based request and response workflows. Frontend API requests are handled through the centralized services/api.js module, which manages request configuration, token handling, and backend communication.

The backend processes incoming requests through middleware, controllers, and service layers before generating structured responses. JSON is used as the primary data exchange format between the frontend and backend systems to support efficient communication and modular workflow management.

4.6. Final Application Screenshots

The final application consists of modules including user authentication, Learn Mode, Evaluate Mode, Doubt Mode, and evaluation history management. The frontend interface provides structured workflows for communication training, response evaluation, and AI-assisted feedback generation.

Screenshots of the final application demonstrate frontend interaction workflows, backend integration, evaluation result rendering, and user-oriented communication training functionalities implemented in the system.

The developed application has also been deployed using cloud hosting platforms for production-level access and testing.

Live Application Access:

https://articulate-devlang.lgcsystems.xyz

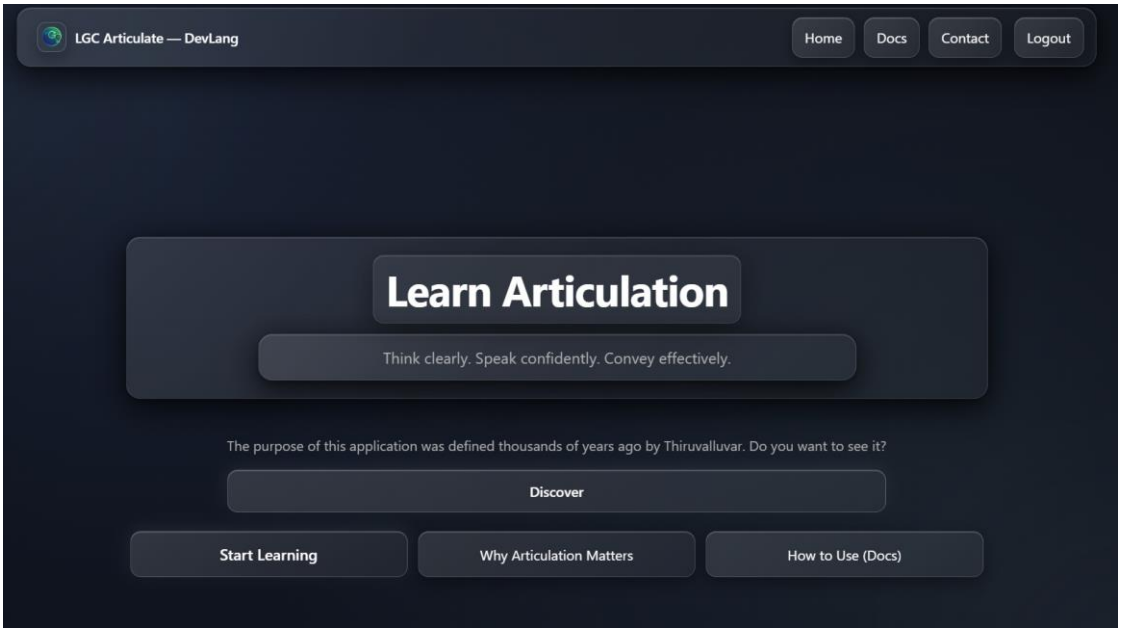

Fig 4.3 Home Page Interface

Illustrates the landing page and primary navigation interface of the developed system.

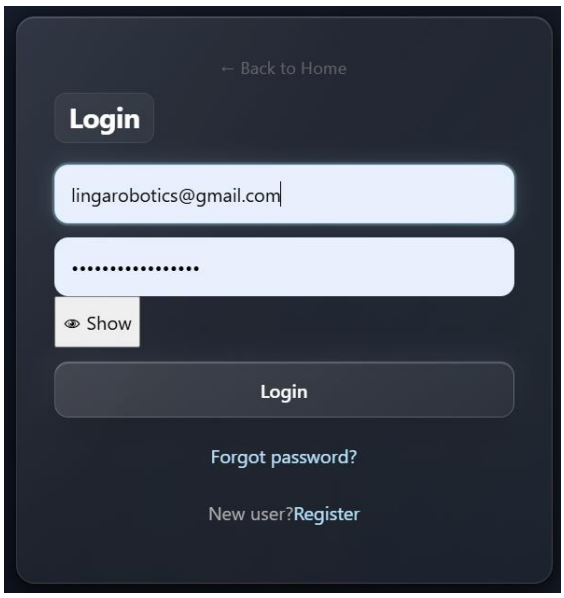

Fig 4.4 User Login Interface

Illustrates the secure login interface used for user authentication and protected module access

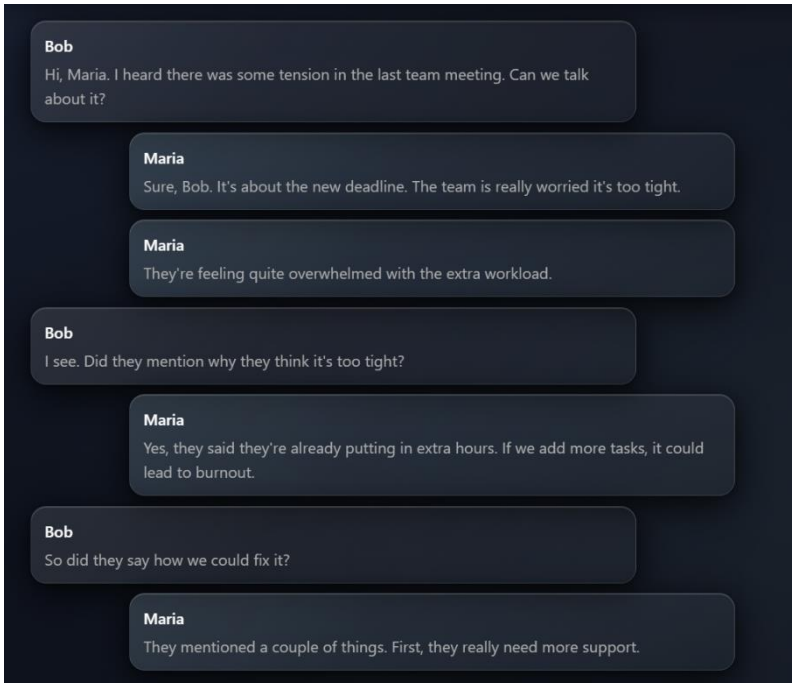

Fig 4.5 Learn Mode Conversation Interface

Illustrates the conversational learning module used for articulation-based communication training.

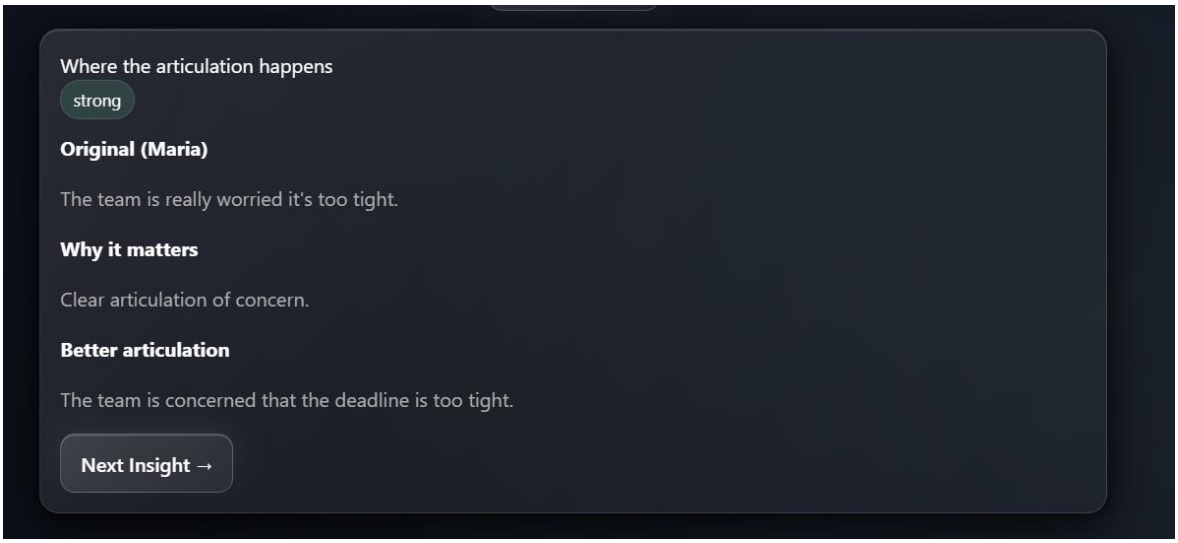

Fig 4.6 Articulation Insight – Strong Communication

Illustrates AI-generated articulation insight for a strongly articulated communication statement.

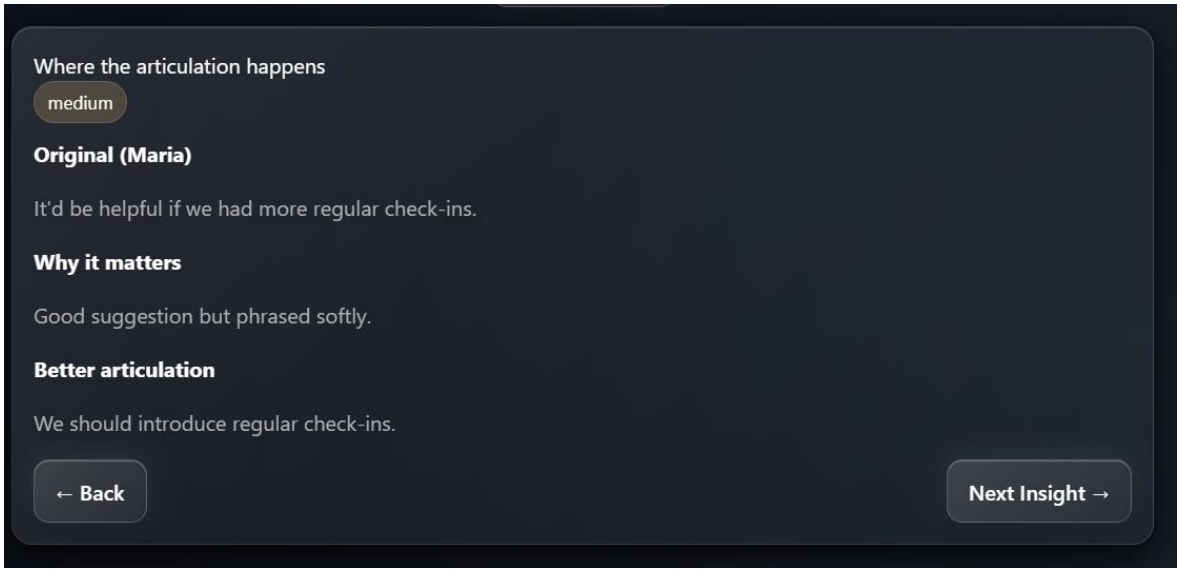

Fig 4.7 Articulation Insight – Medium Communication

Illustrates AI-generated articulation insight for a moderately articulated communication statement.

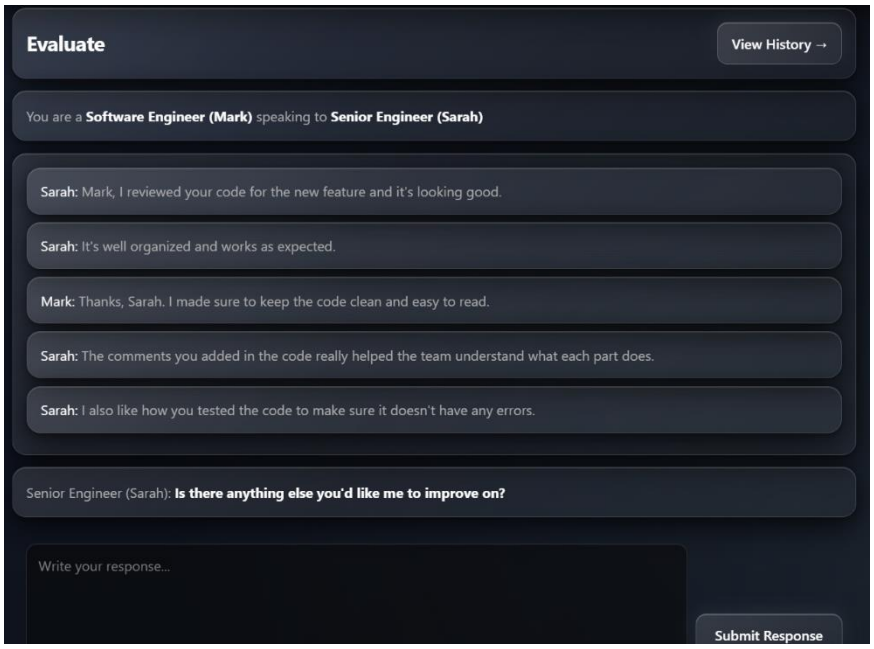

Fig 4.8 Communication Evaluation Input Interface

Illustrates the contextual communication evaluation interface used for articulation assessment.

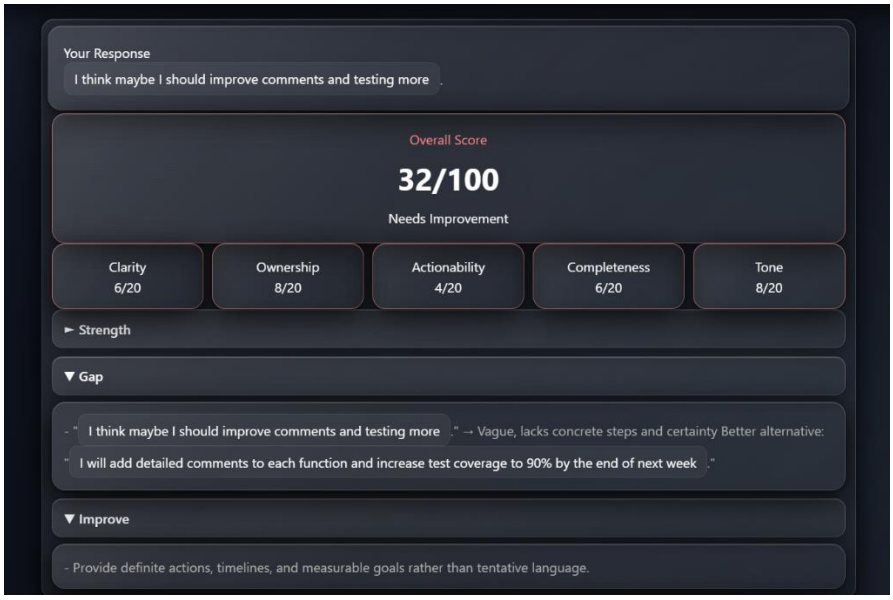

Fig 4.9 AI Evaluation for Weak Articulation Response

Illustrates AI-generated feedback for a weakly articulated professional response.

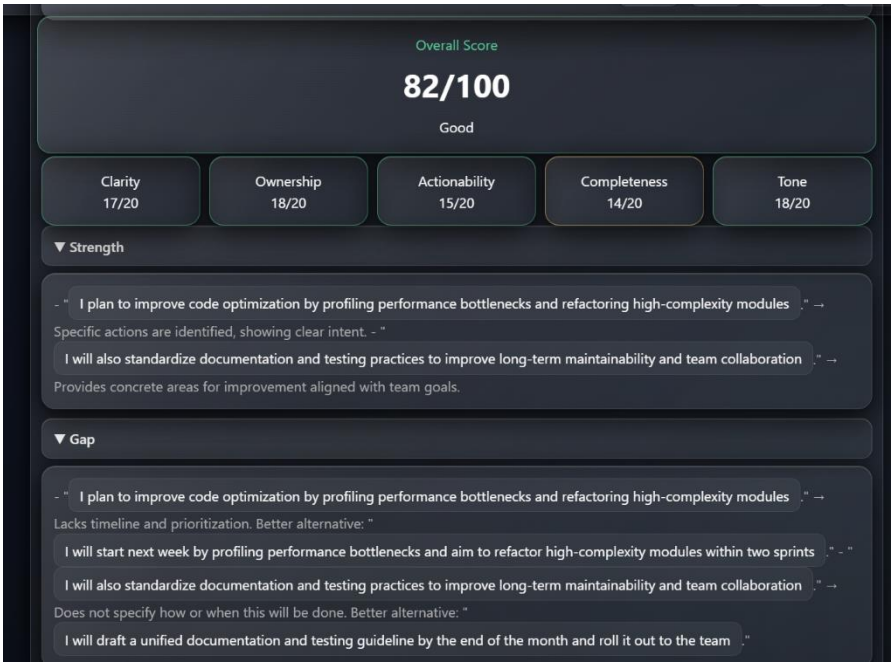

Fig 4.10 AI Evaluation for Structured Articulation Response

Illustrates AI-generated evaluation for a professionally articulated and structured response.

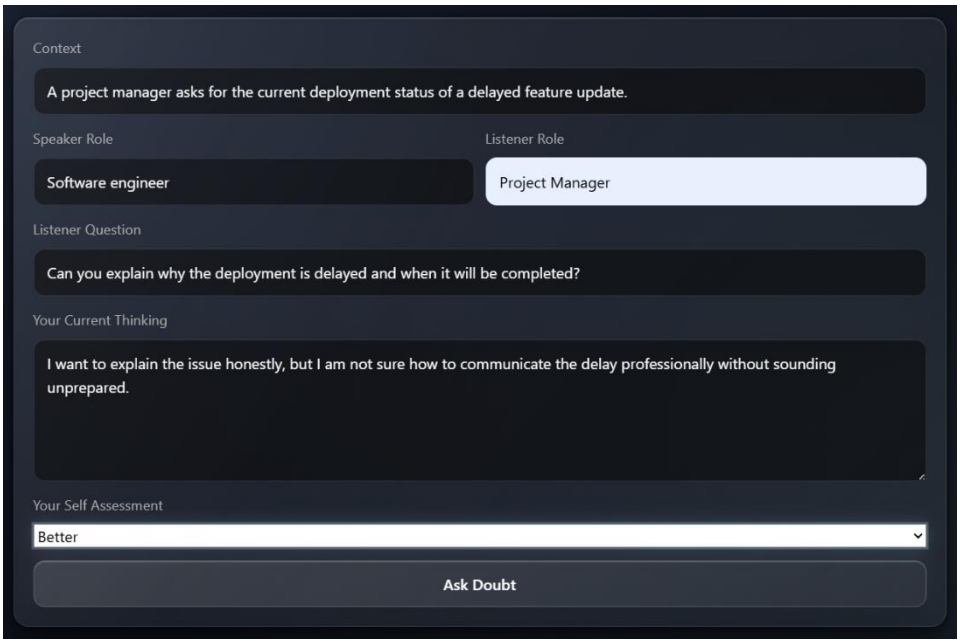

Fig 4.11 Doubt Mode Communication Query Interface

Illustrates the Doubt Mode input interface used for AI-assisted communication guidance.

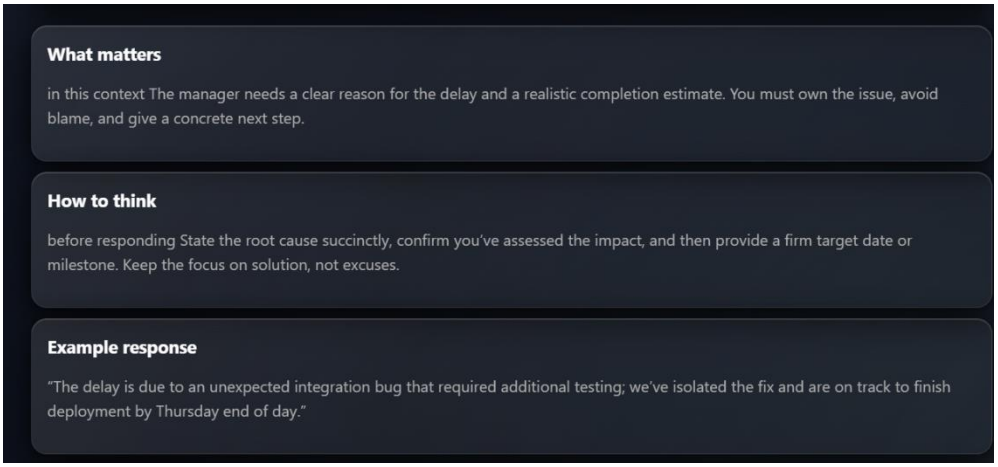

Fig 4.12 AI-Assisted Communication Guidance Response

Illustrates AI-generated communication coaching and contextual articulation guidance.

CHAPTER V

APPLICATIONS AND FUTURE SCOPE

The proposed system can be applied in communication training, professional development, placement preparation, and articulation improvement environments. Engineering students, job seekers, and professionals can use the platform to improve contextual communication, structured thinking, professional response generation, and articulation quality through AI-assisted evaluation and guided communication workflows.

The system can be utilized in educational institutions, placement training programs, soft skill development sessions, and communication-focused learning environments. Features such as Learn Mode, Evaluate Mode, and Doubt Mode support practical communication refinement by helping users understand clarity, ownership, tone, completeness, and professional articulation patterns.

The future scope of the project includes integrating advanced AI models for deeper contextual understanding and adaptive communication evaluation. Future enhancements may include multilingual articulation analysis, voice-based communication assessment, real-time conversational coaching, personalized learning recommendations, and mobile application support.

The system can also be extended with analytics dashboards, organization-specific communication training modules, collaborative learning environments, and expanded professional communication scenarios across different technical and non-technical domains.

CHAPTER VI

COST ESTIMATION

The prototype version of the proposed system was developed using freely available student-accessible technologies and cloud platforms. Most of the software tools, frameworks, hosting services, and development resources used during implementation were available under free-tier usage plans. Existing personal resources such as laptop infrastructure and internet connectivity were utilized during development, resulting in minimal direct development cost for the prototype system.

The current implementation represents a prototype-level deployment intended for academic and experimental usage. However, production-scale deployment with higher traffic handling, advanced AI integrations, enterprise-level infrastructure, enhanced security, and scalable cloud resources may involve additional operational and maintenance costs in future implementations.

Table 6.1

Cost Estimation

S. No	**Resource / Service**	**Platform / Tool**	**Cost**
1	Frontend Hosting	Vercel	Free Tier
2	Backend Hosting	Render	Free Tier
3	AI API Access	OpenRouter / Gemini API	Free Tier

4	Development Environment	VS Code	Free
5	Database Service	MongoDB Atlas	Free Tier
6	Internet Connectivity	Existing Personal Resource	No Additional Cost
7	Laptop Infrastructure	Existing Personal Resource	No Additional Cost

Total Prototype Development Cost: ₹0 (Using Free-Tier and Existing Resources)

CHAPTER VII

CONCLUSION

The project “LGC Articulate: Intelligent Communication Training and Evaluation System” was successfully designed and developed as a full-stack AI-assisted communication training platform focused on articulation improvement, structured thinking, and contextual communication evaluation. The system integrates frontend, backend, database, authentication, and AI orchestration workflows to provide guided communication learning and structured response evaluation.

The developed platform includes modules such as Learn Mode, Evaluate Mode, and Doubt Mode to support communication training, articulation analysis, and AI-assisted communication refinement. Features including JWT authentication, REST API integration, evaluation history management, and production deployment using cloud platforms demonstrate practical implementation of modern full-stack development concepts.

The project also establishes a strong conceptual foundation based on the communication principles expressed in Thirukkural – Porutpaal / Amaichiyal / Avaiyarithal – Kural 711**,** emphasizing contextual understanding and clarity before communication. Through AI-assisted workflows and structured articulation evaluation, the system aims to help users improve professional communication quality, response structure, and communication confidence.

The successful implementation and deployment of the system demonstrate the potential of integrating Artificial Intelligence with communication training workflows to create interactive and scalable articulation learning environments for students and professionals.

REFERENCES

1. React Documentation. https://react.dev/
2. Node.js Official Documentation. https://nodejs.org/
3. Express.js Documentation. https://expressjs.com/
4. MongoDB Documentation. https://www.mongodb.com/docs/
5. JWT Official Website. https://jwt.io/
6. Vite Documentation. https://vitejs.dev/
7. OpenRouter API Documentation. https://openrouter.ai/docs
8. Google Gemini API Documentation. https://ai.google.dev/
9. Vercel Documentation. https://vercel.com/docs
10. Render Documentation. https://render.com/docs
11. Thirukkural – Porutpaal / Amaichiyal / Avaiyarithal – Kural 711